

Guía Room en CatchFood

Cómo se guardó el histórico TOP 10 usando Room sobre SQLite

Idea central para decir en defensa: En CatchFood usé Room para guardar localmente el puntaje final de cada partida. Room genera el código que conecta Kotlin con SQLite, y luego la pantalla de histórico consulta los 10 mejores valores ordenados de mayor a menor.

Esta guía se enfoca únicamente en la persistencia de datos de CatchFood: qué problema resuelve, qué archivos se agregaron, cómo viaja el puntaje desde el juego hasta SQLite y cómo responder preguntas típicas del jurado.

1. Primero: Room, SQLite y PostgreSQL no son lo mismo

Concepto	Qué es	Cómo aplica aquí
SQLite	Base de datos local que vive dentro del teléfono.	Aquí guarda el archivo catch_food.db con los puntajes.
Room	Librería de Android que simplifica el uso de SQLite.	Permite usar @Entity, @Dao y @Database en vez de escribir todo SQL manual.
PostgreSQL	Motor de base de datos de servidor.	No se usa en CatchFood. Sería para backend o nube, no para este histórico local.

Respuesta corta: Room no es PostgreSQL. Room trabaja sobre SQLite local en Android. Lo escogí porque el histórico del juego no necesita internet ni servidor.

2. Archivos donde se aplicó Room

Archivo	Responsabilidad
CatchFoodScoreEntity.kt	Define la tabla catch_food_scores y sus columnas.
CatchFoodScoreDao.kt	Define las operaciones SQL: insertar puntaje y consultar el TOP 10.
CatchFoodDatabase.kt	Crea la base de datos Room y expone el DAO.
CatchFoodScoreRepository.kt	Capa intermedia para que la UI/ViewModel no hable directo con Room.
CatchFoodViewModel.kt	Guarda el puntaje cuando el juego termina.
CatchFoodLeaderboard.kt	Lee los puntajes guardados y los muestra en pantalla.
app/build.gradle.kts	Declara dependencias Room y KSP.

3. Configuración Gradle: dependencias necesarias

En app/build.gradle.kts se agregaron las dependencias de Room y el procesador KSP. KSP genera código en compilación para que Room pueda crear la implementación real de la base de datos y del DAO.

```
plugins {  
    alias(libs.plugins.ksp)  
}  
  
dependencies {  
    implementation(libs.androidx.room.runtime)  
    implementation(libs.androidx.room.ktx)  
    ksp(libs.androidx.room.compiler)  
}
```

- room-runtime: contiene las clases principales de Room.
- room-ktx: agrega soporte cómodo para Kotlin y corrutinas suspend.
- room-compiler con ksp: analiza las anotaciones @Entity, @Dao y @Database, y genera código automáticamente.

4. Entity: la tabla de puntajes

La Entity es la clase que Room convierte en una tabla SQLite. En CatchFood se llama CatchFoodScoreEntity.

```
@Entity(tableName = "catch_food_scores")
data class CatchFoodScoreEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val foodCaught: Int
)
```

Parte	Explicación para defensa
@Entity	Le dice a Room: esta clase representa una tabla.
tableName	Define el nombre real de la tabla en SQLite: catch_food_scores.
data class	Clase de Kotlin pensada para guardar datos. Genera equals, copy, toString, etc.
@PrimaryKey	Marca la columna id como identificador único de cada fila.
autoGenerate = true	SQLite/Room asigna el id automáticamente al insertar.
foodCaught	Es el dato importante: la cantidad de comida que atrapó el jugador.

Cómo se ve como tabla: id sería una columna numérica única; foodCaught sería otra columna numérica con el puntaje final.

5. DAO: las consultas de la base

DAO significa Data Access Object. Es una interfaz donde se declaran las operaciones permitidas contra la base. Room genera la implementación real.

```
@Dao
interface CatchFoodScoreDao {
    @Insert
    suspend fun insert(score: CatchFoodScoreEntity)

    @Query(
        """
        SELECT foodCaught
        FROM catch_food_scores
        ORDER BY foodCaught DESC, id ASC
        LIMIT 10
        """
    )
    suspend fun getTopTenScores(): List<Int>
}
```

- @Dao marca la interfaz como una puerta de acceso a la base.
- @Insert indica que la función insertará una fila en la tabla.
- suspend permite ejecutar la operación dentro de una corrutina, sin bloquear la pantalla del juego.
- @Query contiene SQL puro para consultar la tabla.
- ORDER BY foodCaught DESC ordena de mayor puntaje a menor puntaje.
- id ASC desempata: si dos partidas tienen el mismo puntaje, aparece primero la que se guardó antes.
- LIMIT 10 hace que solo salgan los 10 mejores resultados.

6. Database: creación de Room y patrón Singleton

```
@Database(
    entities = [CatchFoodScoreEntity::class],
    version = 1,
    exportSchema = false
)
abstract class CatchFoodDatabase : RoomDatabase() {
    abstract fun scoreDao(): CatchFoodScoreDao
}
```

La clase CatchFoodDatabase representa la base de datos completa de CatchFood. Hereda de RoomDatabase porque Room necesita una clase central donde se registren las entidades y DAOs.

Elemento	Qué significa
entities	Lista de tablas manejadas por esta base. Aquí solo hay una: CatchFoodScoreEntity.
version = 1	Versión inicial del esquema. Si cambia la estructura de tablas en el futuro, se sube.
exportSchema = false	Evita exportar archivos de esquema. En proyectos grandes puede ponerse true.
abstract fun scoreDao()	Room implementa esta función para devolver el DAO real.

```
companion object {
    @Volatile
    private var instance: CatchFoodDatabase? = null

    fun getInstance(context: Context): CatchFoodDatabase = instance ?: synchronized(this) {
        instance ?: Room.databaseBuilder(
            context.applicationContext,
            CatchFoodDatabase::class.java,
            DATABASE_NAME
        ).build().also { instance = it }
    }
}
```

- @Volatile ayuda a que el valor instance sea visible correctamente entre hilos.
- instance guarda una única copia de la base en memoria.
- synchronized evita que dos hilos creen dos bases al mismo tiempo.
- context.applicationContext evita guardar accidentalmente una Activity y causar fugas de memoria.
- Room.databaseBuilder construye la base SQLite local.
- DATABASE_NAME = "catch_food.db" es el nombre del archivo físico de la base.

7. Repository: capa intermedia limpia

```
class CatchFoodScoreRepository(context: Context) {
    private val scoreDao = CatchFoodDatabase.getInstance(context).scoreDao()

    suspend fun saveScore(score: Int) {
        scoreDao.insert(CatchFoodScoreEntity(foodCaught = score))
    }

    suspend fun getTopScores(): List<Int> = scoreDao.getTopTenScores()
}
```

El Repository existe para que el resto del juego no tenga que conocer detalles de Room. Si mañana se cambia la base local por una API, el cambio se concentra aquí.

- saveScore recibe un Int normal del juego y lo convierte en CatchFoodScoreEntity.
- getTopScores devuelve una lista de Int porque la pantalla solo necesita mostrar el valor foodCaught.
- Ambas funciones son suspend porque terminan llamando operaciones de base de datos.

8. ViewModel: guardar el puntaje al perder

```
private val scoreRepository = CatchFoodScoreRepository(application)
private var scoreSavedForCurrentGame = false

fun updateGame(deltaSeconds: Float) {
    gameLogic.updateGame(deltaSeconds)
    syncUiState()

    if (gameLogic.isGameOver && !scoreSavedForCurrentGame) {
        scoreSavedForCurrentGame = true
        val finalScore = gameLogic.score
        viewModelScope.launch {
            scoreRepository.saveScore(finalScore)
        }
    }
}
```

- El ViewModel detecta cuando `gameLogic.isGameOver` cambia a verdadero.
- `scoreSavedForCurrentGame` evita guardar el mismo resultado muchas veces. Esto es clave porque `updateGame` se ejecuta repetidamente durante el loop.
- `finalScore` copia el puntaje final antes de entrar a la corrutina.
- `viewModelScope.launch` abre una corrutina asociada al ciclo de vida del ViewModel.
- `scoreRepository.saveScore(finalScore)` inserta el registro en SQLite usando Room.

Frase para defensa: Guardé el puntaje desde el ViewModel, no desde el Composable directamente, para separar la lógica del juego y la persistencia de la interfaz visual.

9. Pantalla histórico: leer el TOP 10

```
val context = LocalContext.current
val scoreRepository = remember(context) { CatchFoodScoreRepository(context) }
val scores by produceState(initialValue = emptyList(), scoreRepository) {
    value = scoreRepository.getTopScores()
}
```

- `LocalContext.current` da acceso al Context de Android desde Compose.
- `remember` evita recrear el Repository en cada recomposición innecesaria.
- `produceState` crea un estado de Compose a partir de una operación suspend.
- `initialValue = emptyList()` muestra una lista vacía mientras se carga la consulta.
- `value = scoreRepository.getTopScores()` consulta Room y actualiza la UI automáticamente.

```

LazyColumn {
    itemsIndexed(scores) { index, score ->
        Text(text = "${index + 1}.")
        Text(text = score.toString())
    }
}

```

LazyColumn muestra la lista de puntajes de forma eficiente. itemsIndexed entrega tanto la posición como el valor, por eso se puede enseñar 1, 2, 3... junto al puntaje.

10. Flujo completo del dato

Paso	Qué ocurre	Archivo
1	El jugador juega y consigue comida.	CatchFoodGameLogic.kt
2	El juego termina por veneno o pérdidas.	CatchFoodViewModel.kt
3	El ViewModel detecta isGameOver.	CatchFoodViewModel.kt
4	Se toma gameLogic.score como puntaje final.	CatchFoodViewModel.kt
5	Se llama saveScore en una corrutina.	CatchFoodViewModel.kt
6	El Repository crea CatchFoodScoreEntity.	CatchFoodScoreRepository.kt
7	El DAO inserta la fila en catch_food_scores.	CatchFoodScoreDao.kt
8	SQLite guarda el dato en catch_food.db.	CatchFoodDatabase.kt
9	Histórico consulta SELECT foodCaught ... LIMIT 10.	CatchFoodScoreDao.kt
10	Compose muestra el TOP 10 en pantalla.	CatchFoodLeaderboard.kt

11. Por qué se usó así y no de otra forma

- Se usa Room porque el histórico es local, pequeño y no necesita servidor.
- Se usa SQLite porque ya viene integrado en Android.
- Se usa Repository para separar acceso a datos de la pantalla.
- Se usa ViewModel para que la lógica sobreviva mejor a recomposiciones y no dependa directamente del Composable.
- Se usan corrutinas porque las operaciones de base no deben bloquear el hilo principal.
- Se consulta solo List<Int> porque el requerimiento era mostrar únicamente la comida máxima alcanzada, no fecha ni nombre.

12. Preguntas típicas de defensa

Pregunta	Respuesta recomendada
¿Qué es Room?	Una librería de Android que facilita trabajar con SQLite usando clases, anotaciones y DAOs.
¿Room es PostgreSQL?	No. Room trabaja sobre SQLite local. PostgreSQL es una base de datos de servidor.
¿Dónde se crea la tabla?	En CatchFoodScoreEntity con <code>@Entity(tableName = "catch_food_scores")</code> .
¿Dónde está el SQL?	En CatchFoodScoreDao, dentro de la anotación <code>@Query</code> .
¿Por qué suspend?	Porque insertar o consultar base de datos debe hacerse fuera del hilo principal usando corrutinas.
¿Por qué hay Repository?	Para separar la lógica de datos de la UI y del ViewModel.
¿Qué evita <code>scoreSavedForCurrentGame</code> ?	Evita guardar el mismo puntaje varias veces mientras el estado Game Over sigue activo.
¿Qué pasa si cierro la app?	El dato queda guardado en SQLite porque es persistencia local.

13. Mini guion para explicarlo en 60 segundos

“Para el histórico de CatchFood implementé persistencia local usando Room, que es una capa de Android encima de SQLite. Definí una entidad llamada CatchFoodScoreEntity, que representa la tabla `catch_food_scores` con dos columnas: `id` y `foodCaught`. Luego creé un DAO con dos operaciones: insertar un puntaje y consultar los 10 mejores con SQL ordenado por `foodCaught` descendente. La clase `CatchFoodDatabase` construye la base `catch_food.db` con `Room.databaseBuilder` y usa `Singleton` para no crear múltiples instancias. Encima puse un `Repository` para que el `ViewModel` no dependa directamente de Room. Cuando el juego termina, el `ViewModel` detecta `isGameOver`, toma el score final y lo guarda una sola vez con `viewModelScope.launch`. Finalmente, la pantalla `CatchFoodLeaderboard` usa `produceState` para pedirle al `Repository` el TOP 10 y mostrarlo en `Compose`.”

14. Checklist mental antes de defender

- Room = librería; SQLite = base local; PostgreSQL = servidor externo no usado.
- `@Entity` define tabla.
- `@PrimaryKey(autoGenerate = true)` crea id automático.
- `@Dao` define operaciones.
- `@Insert` inserta.

- @Query ejecuta SQL.
- @Database registra entidades y versión.
- Repository encapsula acceso a datos.
- ViewModel guarda cuando pierde.
- Leaderboard consulta y muestra el TOP 10.

15. Rutas exactas para enseñar en Android Studio

```
app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodScoreEntity.kt
```

```
app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodScoreDao.kt
```

```
app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodDatabase.kt
```

```
app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodScoreRepository.kt
```

```
app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodViewModel.kt
```

```
app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodLeaderboard.kt
```

```
app/build.gradle.kts
```